



Python Functions: From Code to Career

— Complete Cookie Recipe Book —

TechPanda Academy



Cute Turtle Chef's Cookie Recipe: Python List Functions

 Making Cookies
with Python Lists!



Recipe: Cookie List

```
cookie_ingredients = []  
# start with an empty list
```

We build the list step by step!

 **Recipe Goal:**
→ ["choco chip",
oat
sugar cookie"]

Chef Turtle's Note: ☆
Use list functions to mix, sort,
and style your cookies!

append()

Adds item to the **END**
`cookie_ingredients.append("choco chip")`
→ ["choco chip"]

pop()

Removes & **RETURNS** the **LAST** item
`last = cookie_ingredients.pop()`
→ removes "choco chip"

sort()

Sorts the list **A → Z** alphabetically
`cookie_ingredients.sort()`
→ ["choco chip", "oat", "sugar cookie"]

reverse()

Flips the list into **REVERSE** order
`cookie_ingredients.reverse()`
→ ["sugar cookie", "oat", "choco chip"]

insert()

Inserts item at a specific **INDEX**
`cookie_ingredients.insert(1, "oat")`
→ "oat" at index 1

remove()

Removes the **FIRST** matching item
`cookie_ingredients.remove("oat")`
→ "oat" from list

Final Cookie List → ["choco chip", ☆ "sugar cookie"]

Tip: Lists are ordered • changeable • allow duplicates!

 Tip from Chef Turtle: Try these in your Python shell and bake your own list! 

Turtle Chef's Cookie Recipe

Baking Cookies with Python String Functions!



COOKIE RECIPE

INGREDIENTS

- 2 cups flour
- 1 cup butter, softened
- 3/4 cup sugar
- 1 egg
- 1 tsp vanilla extract
- 1/2 tsp baking soda
- Pinch of salt

STEPS

1. Mix dry + wet ingredients
2. Scoop dough onto baking sheet
3. Bake until golden brown
4. Cool & enjoy!

BAKE AT:
350°F (175°C) for 10-12 minutes

PYTHON STRING FUNCTIONS

upper()

Converts all characters to uppercase

```
text = "cookie"  
text.upper() → "COOKIE"
```

lower()

Converts all characters to lowercase

```
text = "COOKIE"  
text.lower() → "cookie"
```

strip()

Removes whitespace from both ends

```
s = " - dough - "  
s.strip() → "dough"
```

split()

Splits string into a list

```
ingredients = "flour,sugar,salt"  
ingredients.split(",") → ["flour", "sugar", "salt"]
```

replace()

Replaces a substring with another

```
recipe = "choco pie"  
recipe.replace("pie", "cookie") → "choco cookie"
```

find() & join()

find() finds substring position • join() joins list into string

```
text = "choco chip"  
text.find("chip") → 6  
"- ".join(["choco", "chip"]) → "choco - chip"
```

Turtle Chef Tip: Strings are immutable! These methods create a new string

Python Set Functions

— Cookie Recipe Guide —

add()



Add a single item to the set.

```
cookies = {"choc", "sugar"}  
cookies.add("salt")  
→ {"choc", "sugar", "salt"}
```

discard()



Remove item; no error if missing.

```
cookies.discard("flour")  
→ {"choc", "salt"} # no error
```

pop()



Remove & return a random item.

```
item = cookies.pop()  
→ "choc" # random; size decreases
```

clear()



Remove all items from the set.

```
cookies.clear()  
→ set() # empty set
```

union()



Combine all items from both sets.

```
a = {"choc", "sugar"}  
b = {"sugar", "salt"}  
a | b → {"choc", "sugar", "salt"}
```

intersection()



Items common to both sets.

```
a = {"choc", "sugar"}  
b = {"sugar", "salt"}  
a & b → {"sugar"}
```

difference()



Items in first set but not second.

```
a - b → {"choc"}
```

symmetric_difference()



Items in either set, not both.

```
a ^ b → {"choc", "salt"}
```

issubset()



True if all items are in the other set.

```
{"choc"}.issubset(a) → True
```

issuperset()



True if set contains all items of the other.

```
a.issuperset("choc") → True
```

Chef's Tip: Sets are perfect for removing duplicates, checking membership, and doing math-like comparisons! Use them to keep your cookie mix tidy and unique!

Turtle Chef's Cookie Recipe:

Python Dictionary Functions



Recipe: Cookie Jar Dictionary!

```
cookies = {"choco_chip": 12, "oatmeal": 8, "snickerdoodle": 5}
```

→ A dictionary is like a labeled jar of cookies! 🍪

🍪 **get()**

Safely get a value by key

```
cookies.get("choco_chip")  
cookies.get("macaron", 0)
```

Returns default if key missing

🔑 **keys()**

Get all the keys

```
cookies.keys() # → dict_keys([...])  
list(cookies.keys()) → ["choco_chip",  
"oatmeal", "snickerdoodle"]
```

🍪 **values()**

Get all the values

```
cookies.values()  
# → dict_values([...])  
list_values))) → [12, 8, 5]
```

🔗 **items()**

Get key-value pairs

```
for k, v in cookies.items():  
    print(k, v) # choco_chip 12  
               # oatmeal 8, snickerdoodle 5
```

🥄 **pop()**

Remove & return a value

```
cookies.pop("oatmeal")  
# → 8
```

removes "oatmeal", now 2 items left

+ **update()**

Update or add pairs

```
cookies.update("macaron": 6))  
# adds macaron: 6 to dict
```

🗑️ **clear()**

`cookies.clear()` # → {} | Empty the jar! Clears all items for a fresh batch

Tip: Dictionaries store data as key → value pairs • Like labeled cookie jars!
Keep your cookie recipe organized 💡

Cute Turtle Chef's Cookie Recipe

— Python User Defined Functions —
A Friendly Guide to def, Parameters, Return & Function Call

Recipe Analogy:

Baking Cookies = Writing a Function!

1. def keyword Define Your Function

def is how you create a new function —
like writing your recipe!

```
def bake_cookie(ingredients, temp):
```

- def = tells Python: we are defining a function now

2. Parameters vs Arguments

Parameters =
the placeholders
inside the
function
definition

parameters → (ingredients, temp)
• written in the def line

Arguments = the actual values you
pass when you call the function

arguments → (["flour", "sugar"], 180)
• the real data you supply

3. return statement Give the Result Back

return = sends the result back from the function

```
baked_cookies = mix_and_bake(ingredients, temp).  
return baked_cookies
```

- Without return, the function produces nothing → None

4. Function Call Run the Function!

Call the function to use it:

```
cookies = bake_cookie("flour", "sugar"), 180)  
→ Output: "Freshly baked cookies! 🍪"
```

- The function runs and returns the baked cookies!

Tip for Python Chefs: Functions are reusable 🍪
call them with new ingredients anytime!

Why use functions? • Reusable • Organized • Easier to Test • Clean & Clear Code

Turtle Chef's Cookie Recipe:

Python Built-in Functions



Bake perfect cookies using these 12 Python built-ins! ✨

🥄 Ingredients: 12 Built-in Functions 🍪



print()

`print("Bake!")` → displays
Bake! in the console



input()

`input("How many?")` → gets
user input from keyboard



len()

counts items in list, e.g.
`len([choc, sugar])` → 2



type()

returns the data type, e.g.
`type(5)` → `<class 'int'>`

int()

`int("12")` →
converts to integer
→ 12

str()

`str(100)`
→ converts to text
→ "100"

max()

`max([10,5,8])`
returns largest
→ 10

min()

`min([10,5,8])`
returns smallest
→ 5



sum()

`sum([1,2,3])`
→ adds all
→ 6



round()

`round(2.756,1)`
rounds to 1 decimal
→ 2.8



range(5)

`range(5)`
generates 0,1,2,3,4



sorted()

`sorted([5,2,9])`
→ sorts list
→ [2,5,9]

Recipe Steps 🍪

1. Mix ingredients

use `int()` & `str()` to
prep your data

2. Bake & check

use `len()`, `max()`, `min()`, `sum()`,
`round()` to test your batch

3. Serve!

use `print()` to share results &
`sorted()` to arrange cookies!

Try it out!

```
cookies = [5, 2, 9]
total = sum(cookies) # → 16
avg = round(total / len(cookies), 1) # → 5.3
print(f"Baked {len(cookies)} cookies! Avg: {avg}")
```

Tip: All these are Python built-in functions — no import needed! Just use and bake. 🐢 ✨

Sweet Byte Bakery • Learn Python by Baking!